



BLM4522

Ağ Tabanlı Paralel Dağıtım Sistemleri

PROJE RAPORU

(Final Ödevi)

Kapsanan Projeler

- 1) Veritabanı Yedekleme ve Felaketten Kurtarma Planı
- 2) Veri Temizleme ve ETL Süreçleri Tasarımı
- 3) Veritabanı Yedekleme ve Otomasyon Çalışması

Öğrenci	Furkan Yılmaz
Öğrenci No	22290618
Tarih	Haziran 2026
GitHub	github.com/furkanyilmaz/BLM4522

Veritabanı Yedekleme ve Felaketten Kurtarma Planı Video :
<https://drive.google.com/file/d/19qkrqasKiD-mHJ9Yx4D0b32B9KWteyge/view?usp=sharing>

Veri Temizleme ve ETL Süreçleri Tasarımı Video:
<https://drive.google.com/file/d/1VgWldDYUtOjI2ovypIRCTSmZqDo3cCdo/view?usp=sharing>

Veritabanı Yedekleme ve Otomasyon Çalışması Video:
https://drive.google.com/file/d/16_dGzqTCiTsIqcfUI6Qu8rjZ4UKIKVmL/view?usp=sharing

İçindekiler

Genel Giriş.....	3
1. Veritabanı Yedekleme ve Felaketten Kurtarma Planı	4
1.1. Giriş ve Proje Amacı.....	4
1.2. Kullanılan Teknolojiler	4
1.3. Demo Veritabanı ve Örnek Veri	4
1.4. Yedekleme Stratejileri (Tam / Fark / Artık)	4
1.5. Belirli Bir Ana Geri Dönüş (Point-in-Time Recovery).....	5
1.6. WAL Arşivleme Yapılandırması.....	6
1.7. Gerçek Yedek Alma (pg_dump).....	6
1.8. Felaket Senaryosu ve Kurtarma.....	7
1.9. Streaming Replication ve Veritabanı Yansıtma (Mirroring)	7
1.10. Zamanlanmış Yedekleme.....	8
1.11. SQL Server – PostgreSQL Kavram Karşılıkları.....	8
1.12. Sonuç ve Değerlendirme.....	8
2. Veri Temizleme ve ETL Süreçleri Tasarımı.....	9
2.1. Giriş ve Proje Amacı.....	9
2.2. Kullanılan Teknolojiler ve Katmanlı Mimari	9
2.3. EXTRACT – Kirliliği Ham Veri.....	9
2.4. Veri Profillemesi ve Hata Tespiti	10
2.5. CLEANING – Veri Temizleme	10
2.6. TRANSFORM – Para Birimi Dönüştürme	11
2.7. Reddedilen Kayıtlar	11
2.8. Önce / Sonra Kalite Karşılaştırması	12
2.9. ETL Log ve Başarı Oranı.....	12
2.10. Idempotent Yükleme (UPSERT)	13
2.11. Türkçe Karakter Sorunu ve Çözümü	13
2.12. Sonuç ve Değerlendirme.....	13
3. Veritabanı Yedekleme ve Otomasyon Çalışması	14
3.1. Giriş ve Proje Amacı.....	14
3.2. Tanımlı Yedekleme İşleri.....	14
3.3. Job Çalıştırma Prosedürü ve Hata Yakalama	14
3.4. Günlük Yedekleme Raporu	15
3.5. Otomatik Zamanlama Üretimi.....	15
3.6. SLA Denetimi	16
3.7. Otomatik Uyarılar (Trigger).....	16
3.8. Genel Denetim Özeti	17
3.9. Gerçek Job Runner Scripti	17
3.10. SQL Server Agent – PostgreSQL Karşılıkları.....	18
3.11. Sonuç ve Değerlendirme.....	19
Genel Sonuç ve Değerlendirme.....	20

Genel Giriş

Bu rapor, BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında geliştirilen üç ayrı projeyi bir arada sunmaktadır. Projelerin tamamı, bir veritabanı yönetim yaşam döngüsünün birbirini tamamlayan üç temel halkasını ele alır: verinin **yedeklenmesi ve felaketten kurtarılması**, verinin **temizlenmesi ve dönüştürülmesi (ETL)** ve bu işlemlerin **otomatikleştirilip denetlenmesi**.

Tüm projeler ortak bir teknik altyapı üzerinde gerçekleştirilmiştir. Veritabanı yönetim sistemi olarak **PostgreSQL 14** (macOS / Homebrew kurulumu), arayüz olarak **psql** komut satırı kullanılmıştır. SQL dosyaları Türkçe açıklamalı ve numaralandırılmış (00, 01, 02 ...) biçimde hazırlanmış, her proje kendi katmanlı yapısı içinde çalıştırılmıştır. Raporun ilerleyen bölümlerinde her proje; amacı, kullanılan teknolojiler, uygulanan adımlar ve elde edilen çıktılar (ekran görüntüleri) ile ayrıntılı olarak açıklanmaktadır.

1. Veritabanı Yedekleme ve Felaketten Kurtarma Planı

1.1. Giriş ve Proje Amacı

Bu projenin amacı, bir PostgreSQL veritabanı için uçtan uca bir yedekleme ve felaketten kurtarma planı tasarlamak ve bu planın çalıştığını gerçek komutlarla kanıtlamaktır. Proje kapsamında yedekleme stratejileri (tam, fark, artık), belirli bir ana geri dönüş (Point-in-Time Recovery), WAL arşivleme, streaming replication (veritabanı yansıtma) ve zamanlanmış yedekleme konuları ele alınmıştır. Testler için müşteri ve sipariş verisi içeren **backup_demo** adlı bir demo veritabanı oluşturulmuştur.

1.2. Kullanılan Teknolojiler

Projede kullanılan başlıca araçlar ve görevleri aşağıdaki tabloda özetlenmiştir:

Teknoloji / Araç	Görevi
PostgreSQL 14	İlişkisel veritabanı yönetim sistemi
psql	Komut satırı veritabanı istemcisi
pg_dump	Mantıksal (logical) yedek alma aracı
pg_restore	Custom/dizin formatındaki yedekleri geri yükleme aracı
pg_basebackup	Fiziksel temel (base) yedek alma aracı
WAL (Write-Ahead Log)	Değişiklik kayıtları; PITR ve replikasyonun temeli

1.3. Demo Veritabanı ve Örnek Veri

Yedekleme senaryolarının üzerinde uygulanacağı **backup_demo** veritabanı, müşterileri tutan customers ve siparişleri tutan orders tablolarından oluşur. Başlangıç durumunda tablolardaki kayıt sayıları aşağıdaki sorgu ile doğrulanmıştır.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT 'customers' AS tablo, COUNT(*)
FROM customers UNION ALL SELECT 'orders', COUNT(*) FROM orders;"

```

tablo	count
customers	5
orders	7

(2 rows)

Şekil 1: backup_demo veritabanındaki örnek veri (customers = 5, orders = 7 satır)

Şekil 1'de görüldüğü gibi başlangıçta customers tablosunda 5, orders tablosunda 7 kayıt bulunmaktadır. Bu sayılar, ilerleyen bölümlerdeki felaket ve kurtarma adımlarının başarısını ölçmek için referans olarak kullanılacaktır.

1.4. Yedekleme Stratejileri (Tam / Fark / Artık)

Sağlıklı bir yedekleme planı, farklı sıklık ve maliyetlere sahip stratejilerin bir arada kullanılmasına dayanır. Projede dört strateji tanımlanmış ve bir tabloda saklanmıştır.

Tam, Fark ve Artık Yedek Nedir?

Tam (Full) Yedek: Veritabanının o andaki tamamının kopyasıdır. En kapsamlı ama en maliyetli yedektir; PostgreSQL'de `pg_dump` veya `pg_basebackup` ile alınır.

Fark (Differential) Yedek: Son tam yedekten bu yana değişen verileri içerir. PostgreSQL'de bu davranış, son tam yedekten itibaren biriken WAL arşivi ile sağlanır.

Artık (Incremental) Yedek: Bir önceki yedekten (tam ya da artık) bu yana değişenleri içerir. PostgreSQL'de sürekli WAL akışı (streaming) bu role karşılık gelir ve en küçük veri kaybı (RPO) hedefini sağlar.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT yedek_tipi, siklik, yontem FROM backup_strategy ORDER BY id;"
```

yedek_tipi	siklik	yontem
Tam	Haftalik (Pazar 02:00)	pg_dump / pg_basebackup
Fark	Gunluk (her gece 02:00)	WAL arşivi (son tam yedekten beri)
Artık	Saatlik / surekli	WAL streaming + archive
PITR	Surekli	base backup + WAL replay

(4 rows)

Şekil 2: Tam / Fark / Artık ve PITR stratejilerinin sıklık ve PostgreSQL yöntemleriyle birlikte gösterimi

Şekil 2'deki tabloya göre Tam yedek haftalık (Pazar 02:00, `pg_dump` / `pg_basebackup`), Fark yedek günlük (her gece 02:00, WAL arşivi), Artık yedek saatlik/sürekli (WAL streaming + archive) ve PITR ise sürekli (base backup + WAL replay) olarak planlanmıştır. Bu kademeli yapı, hem depolama maliyetini düşürür hem de olası bir kayıpta geri dönülebilecek noktaları sıklaştırır.

1.5. Belirli Bir Ana Geri Dönüş (Point-in-Time Recovery)

PITR Nedir?

Point-in-Time Recovery (PITR), bir veritabanını geçmişteki belirli bir ana geri döndürme tekniğidir. Temeli, bir tam (base) yedeğin üzerine, o ana kadar üretilmiş WAL kayıtlarının yeniden oynatılmasına (replay) dayanır.

Böylece örneğin yanlışlıkla yapılan bir silme işleminden hemen önceki saniyeye geri dönmek mümkün olur; hedef nokta zaman, işlem (transaction) numarası veya WAL pozisyonu ile belirtilebilir.

PostgreSQL'de geri dönülecek hedef, aşağıdaki parametrelerden biriyle belirlenir:

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT parametre, aciklama, ornek FROM pitr_recovery_targets;"
```

parametre	aciklama	ornek
<code>recovery_target_time</code>	Belirli bir zamana geri don	2026-06-04 14:30:00
<code>recovery_target_xid</code>	Belirli bir islem (transaction) ID	123456
<code>recovery_target_lsn</code>	Belirli bir WAL pozisyonu (LSN)	0/3000000
<code>recovery_target_name</code>	Önceden işaretlenmiş restore point	silme_öncesi
<code>recovery_target</code>	En son tutarlı ana kadar (immediate)	immediate

(5 rows)

Şekil 3: PITR hedef parametreleri: `recovery_target_time` / `xid` / `lsn` / `name` / `immediate`

Şekil 3'te görülen seçenekler şunlardır: `recovery_target_time` belirli bir zamana, `recovery_target_xid` belirli bir işlem numarasına, `recovery_target_lsn` belirli bir WAL pozisyonuna, `recovery_target_name` önceden işaretlenmiş bir restore noktasına, `recovery_target`

= immediate ise en son tutarlı ana kadar kurtarmayı sağlar. Örnekte `silme_onesi` adlı bir restore noktası, felaket senaryosundan önce işaretlenecek bir geri dönüş noktası olarak tasarlanmıştır.

1.6. WAL Arşivleme Yapılandırması

PITR ve fark yedeğinin çalışabilmesi için WAL kayıtlarının kalıcı olarak arşivlenmesi gerekir. Bu, `postgresql.conf` dosyasındaki birkaç ayar ile etkinleştirilir:

```
# WAL arşivlemeyi etkinleştir (postgresql.conf)
wal_level = replica          # replikasyon ve PITR için yeterli seviye
archive_mode = on            # WAL arşivleme açık
archive_command = 'cp %p /arsiv/wal/%f' # her WAL dosyasını arşive kopyala
max_wal_senders = 5          # standby sunuculara akış için
-- Önceden geri dönüş noktası işaretle:
SELECT pg_create_restore_point('silme_onesi');
```

Burada `archive_command` her dolan WAL segmentini güvenli bir arşiv dizinine kopyalar; `pg_create_restore_point` ise daha sonra PITR ile dönülebilecek adlandırılmış bir nokta oluşturur.

1.7. Gerçek Yedek Alma (pg_dump)

pg_dump Custom Format Nedir?

`pg_dump`'ın custom (-Fc) formatı, sıkıştırılmış ve seçici geri yüklemeye uygun ikili bir yedek üretir. Bu format sayesinde tek bir tablo bile ayrı ayrı geri yüklenebilir ve `pg_restore --list` ile yedeğin içeriği denetlenebilir.

Demo veritabanının yedeği custom formatında alınmış ve yedeğin bütünlüğü içerik listesiyle doğrulanmıştır:

```
# Custom formatta yedek al
pg_dump -U postgres -Fc backup_demo -f backups/before_disaster.dump
# Yedeğin içeriğini (bütünlüğünü) listele
pg_restore --list backups/before_disaster.dump
```

```
Yedek basariyla alindi:
-rw-r--r--@ 1 furkanyilmaz staff 13K 4 Jun 20:03 backup_sql/back
ups/before_disaster.dump
3821; 0 35015 TABLE DATA public backup_history postgres
3823; 0 35028 TABLE DATA public backup_strategy postgres
3817; 0 34990 TABLE DATA public customers postgres
3819; 0 35000 TABLE DATA public orders postgres
3824; 0 35041 TABLE DATA public pitr_recovery_targets postgres
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 %
```

Şekil 4: Yedeğin başarıyla alınması (13K dump dosyası) ve `pg_restore --list` ile tablo listesinin doğrulanması

Şekil 4, yedeğin başarıyla oluşturulduğunu (before_disaster.dump, 13K) ve içerisinde customers, orders, backup_history gibi tabloların eksiksiz yer aldığını göstermektedir. Bu liste, yedeğin geri yükleme öncesinde bozuk olmadığının kanıtıdır.

1.8. Felaket Senaryosu ve Kurtarma

Planın gerçekten işe yaradığını göstermek için kontrollü bir felaket canlandırılmıştır: orders tablosundaki tüm kayıtlar kazara silinmiş gibi davranılarak silinmiştir.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "DELETE FROM orders; SELECT COUNT(*) AS kalan_siparis FROM orders;"
```

kalan_siparis
0

(1 row)

Şekil 5: Felaket senaryosu: orders tablosundaki tüm kayıtların silinmesi (kalan_siparis = 0)

Şekil 5'te DELETE FROM orders komutu sonrasında tabloda kalan kayıt sayısı 0'a düşmüştür. Bu, üretimde yaşanabilecek kazara veri kaybını temsil eder. Ardından önceden alınan yedek kullanılarak yalnızca etkilenen tablo geri yüklenmiştir.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % pg_restore -U postgres -d backup_demo -t orders --data-only backup_sql/backups/before_disaster.dump; psql -U postgres -d backup_demo -c "SELECT COUNT(*) AS geri_gelen_siparis FROM orders;"
```

geri_gelen_siparis
7

(1 row)

Şekil 6: Felaketten kurtarma: pg_restore ile yalnızca orders tablosunun geri yüklenmesi (geri_gelen_siparis = 7)

Şekil 6, pg_restore --data-only -t orders komutuyla yalnızca sipariş verisinin geri yüklendiğini ve kayıt sayısının yeniden 7'ye ulaştığını göstermektedir. Şekil 5 ve Şekil 6 birlikte, **felaket** → **kurtarma** döngüsünün uçtan uca çalıştığının somut kanıtıdır: silinen 7 sipariş, hiçbir veri kaybı olmadan yedekten geri getirilmiştir.

1.9. Streaming Replication ve Veritabanı Yansıtma (Mirroring)

Streaming Replication Nedir?

Streaming Replication, birincil (primary) sunucudaki WAL kayıtlarının gerçek zamanlı olarak bir veya daha fazla yedek (standby) sunucuya akıtılmasıdır. SQL Server'daki Database Mirroring kavramının PostgreSQL karşılığıdır.

Senkron modda işlem, standby de yazana kadar onaylanmaz (sıfır veri kaybı); asenkron modda ise birincil beklemes (daha yüksek başarımlı). Birincil sunucu çökerse standby, failover ile devreye alınabilir.

Bir standby sunucu, birincilin temel yedeğinden başlatılır ve sürekli WAL akışını dinler:

```
# Standby için temel yedek al ve akışı başlat
pg_basebackup -h primary_host -U replicator -D /var/lib/pg/standby -R
-- Birincil sunucuda replikasyon kullanıcısı:
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'gizli';
```

1.10. Zamanlanmış Yedekleme

Yedeklerin elle alınması güvenilir değildir; bu nedenle yedekler zamanlanmış görevlerle otomatikleştirilir. İşletim sistemi seviyesinde crontab, veritabanı içinde ise pg_cron eklentisi kullanılabilir.

```
# crontab: her gün 02:00'de tam yedek
0 2 * * * pg_dump -U postgres -Fc backup_demo -f /backups/daily_$(date +%F).dump
-- pg_cron ile veritabanı içinden zamanlama:
SELECT cron.schedule('gunluk_tam', '0 2 * * *', $$CALL sp_run_backup_job(1)$$);
```

1.11. SQL Server – PostgreSQL Kavram Karşılıkları

Projede ele alınan kavramların SQL Server ve PostgreSQL dünyasındaki karşılıkları aşağıda eşleştirilmiştir:

SQL Server Kavramı	PostgreSQL Karşılığı
Full Backup	pg_dump / pg_basebackup
Differential Backup	WAL arşivi (son tam yedekten beri)
Transaction Log Backup	WAL streaming / archive
Point-in-time Restore	PITR (recovery_target_time)
Database Mirroring	Streaming Replication (primary–standby)
SQL Server Agent (zamanlama)	pg_cron / crontab

1.12. Sonuç ve Değerlendirme

Bu projede bir PostgreSQL veritabanı için kademeli bir yedekleme stratejisi (tam/fark/artık/PITR) tasarlanmış, gerçek bir yedek alınmış, kontrollü bir felaket canlandırılmış ve veri başarıyla kurtarılmıştır. Silinen 7 siparişin eksiksiz geri getirilmesi, planın yalnızca teoride değil pratikte de çalıştığını göstermektedir. Ayrıca streaming replication ve zamanlanmış yedekleme ile sürekliliğin nasıl sağlanacağı ortaya konmuştur.

2. Veri Temizleme ve ETL Süreçleri Tasarımı

2.1. Giriş ve Proje Amacı

Bu projenin amacı, iki farklı kaynaktan gelen "kirli" müşteri ve satış verisini bir ETL süreciyle temizleyip dönüştürmek, hedef veri ambarına yüklemek ve sürecin veri kalitesine etkisini raporlamaktır. Gerçek hayatta veriler nadiren tertemiz gelir; tutarsız şehir adları, büyük/küçük harf karmaşası, bozuk tarih formatları ve eksik alanlar sık karşılaşılan sorunlardır.

ETL Nedir?

ETL (Extract – Transform – Load), verinin kaynaklardan çekilmesi (Extract), iş kurallarına göre temizlenip dönüştürülmesi (Transform) ve hedef ambara yüklenmesi (Load) sürecidir.

Bu projede süreç, katmanlı bir mimari üzerinde yürütülür: ham verinin tutulduğu staging katmanı, temizlenmiş verinin tutulduğu clean katmanı ve nihai analize hazır verinin yüklendiği target katmanı.

2.2. Kullanılan Teknolojiler ve Katmanlı Mimari

Proje, veri kalitesini adım adım iyileştiren üç katmanlı bir mimari üzerine kuruludur:

Katman	Görevi
staging (ham)	Kaynaklardan gelen ham verinin hiç dokunulmadan tutulduğu alan
clean (temiz)	Temizleme ve dönüştürme kurallarının uygulandığı ara katman
target (hedef)	Analize hazır, kaliteli verinin ve kalite raporlarının yüklendiği ambar

2.3. EXTRACT – Kirli Ham Veri

Süreç, online_sales gibi farklı kaynaklardan gelen verinin staging katmanına olduğu gibi alınmasıyla başlar. Bu aşamada veriye müdahale edilmez; amaç, kaynaktaki kalite sorunlarının görünür olmasıdır.

```
SELECT muster_i_adi, eposta, sehir, birim_fiyat, satis_tarihi FROM staging.online_sales LIMIT 8;
```

muster_i_adi	eposta	sehir	birim_fiyat	satis_tarihi
Ahmet Yılmaz	ahmet@example.com	İstanbul	25000	2024-01-15
Ayşe Demir	AYSE@EXAMPLE.COM	istanbul	350	2024-01-16
Mehmet Kaya	mehmet@example.com	ISTANBUL	850	15.01.2024
Fatma Şahin	fatma@example.com	ankara	4500	2024/01/17
Can Öztürk	can@example.com	Ankara	1200	2024-01-18
Zeynep Acar		İzmir	950	2024-01-19
Burak Çelik		izmir	180	2024-01-20
Elif Yıldız	elif@example.com	Bursa	25000	2024-01-21

(8 rows)

Şekil 7: *EXTRACT* aşaması: *staging* katmanındaki kirli ham veri (tutarsız şehir, büyük harf e-posta, karışık tarih, boş alanlar)

Şekil 7'de tipik kalite sorunları açıkça görülmektedir: aynı şehir farklı yazımlarla ("İstanbul", "istanbul", "ISTANBUL"), e-postalar büyük harfle (AYSE@EXAMPLE.COM), tarihler birbirinden farklı formatlarda (2024-01-15, 15.01.2024, 2024/01/17) ve bazı e-posta alanları boş şekilde gelmiştir. Bu sorunlar, sonraki adımların çözmesi gereken hedeflerdir.

2.4. Veri Profillemeye ve Hata Tespiti

Temizleme öncesinde verinin kalitesi ölçülür. Bu adımda her hata türü sayılarak bir profil çıkarılır; böylece temizlemenin etkisi daha sonra nicel olarak karşılaştırılabilir.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d etl_demo -c "
SELECT metrik, deger FROM target.quality_profile WHERE asama='ONCE' ORDER BY
metrik;"
```

metrik	deger
-----	-----
eksik_eposta	3
eksik_sehir	1
gecersiz_adet	1
gecersiz_eposta_format	1
negatif_fiyat	1
sifir_adet	1
toplam_ham_kayit	22
(7 rows)	

Şekil 8: Veri profillemeye: 22 ham kayıt üzerinde tespit edilen hata türleri ve adetleri

Şekil 8'e göre toplam 22 ham kayıta 3 eksik e-posta, 1 eksik şehir, 1 geçersiz adet, 1 geçersiz e-posta formatı, 1 negatif fiyat ve 1 sıfır adet tespit edilmiştir. Bu profil, hangi kuralların uygulanması gerektiğini ve kaç kaydın etkilendiğini netleştirir.

2.5. CLEANING – Veri Temizleme

Tespit edilen sorunlar, *clean* katmanında iş kuralları uygulanarak giderilir: e-postalar küçük harfe çevrilir, telefon numaraları +90 formatına standartlaştırılır, şehir adları Türkçe karakterlere duyarlı biçimde düzeltilir ve tarihler tek bir formata (YYYY-MM-DD) getirilir.

```
-- Temizleme kuralları (clean.sales_clean)
UPDATE clean.sales_clean SET
  eposta = lower(trim(eposta)),          -- küçük harf + boşluk temizle
  sehir = tr_initcap(trim(sehir)),       -- Türkçe duyarlı baş harf
  telefon = '+90' || right(regexp_replace(telefon, '\D', '', 'g'), 10);
```

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d etl_demo -c "SELECT ad, e
posta, telefon, sehir, fiyat, satis_tarihi FROM clean.sales_clean ORDER BY id LIMIT 8;"
```

ad	eposta	telefon	sehir	fiyat	satis_tarihi
Ahmet Yılmaz	ahmet@example.com	+905321112233	İstanbul	25000.00	2024-01-15
Ayşe Demir	ayse@example.com	+905322223344	İstanbul	350.00	2024-01-16
Mehmet Kaya	mehmet@example.com	+905333334455	İstanbul	850.00	2024-01-15
Fatma Şahin	fatma@example.com	+905344445566	Ankara	4500.00	2024-01-17
Can Öztürk	can@example.com	+905355556677	Ankara	1200.00	2024-01-18
Zeynep Acar		+905366667788	İzmir	950.00	2024-01-19
Burak Çelik		+905377778899	İzmir	180.00	2024-01-20
John Smith	john@example.com	+901555123456	İstanbul	150.00	2024-01-23

(8 rows)

Şekil 9: **CLEANING** aşaması: aynı verilerin temizlenmiş hâli (küçük harf e-posta, +90 telefon, standart şehir ve tarih)

Şekil 9, Şekil 7 ile karşılaştırıldığında temizlemenin etkisi nettir: tüm İstanbul kayıtları artık tek biçimde "İstanbul" olmuş, e-postalar küçük harfe inmiş, telefonlar +90 formatına gelmiş ve tarihler YYYY-MM-DD standardına oturmuştur. Yani **önce kirli** → **sonra temiz** dönüşümü gözle görülür biçimde gerçekleşmiştir.

2.6. TRANSFORM – Para Birimi Dönüştürme

Dönüştürme adımında veriler standartlaştırılır ve türetilmiş sütunlar üretilir. Bu projede farklı para birimindeki satışlar ortak bir birime (TRY) çevrilmiş ve toplam tutar hesaplanmıştır.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d etl_demo -c "SELECT ad, urun, birim_fiyat_orij
inal, orijinal_para_birimi, birim_fiyat_try, toplam_tutar_try FROM clean.sales_transformed WHERE orijinal_para
_birimi='USD';"
```

ad	urun	birim_fiyat_orijinal	orijinal_para_birimi	birim_fiyat_try	toplam_tutar_try
John Smith	Monitör	150.00	USD	4875.00	4875.00
Emma Brown	Laptop	780.00	USD	25350.00	25350.00

(2 rows)

Şekil 10: **TRANSFORM** aşaması: USD cinsinden satışların TRY'ye dönüştürülmesi ve toplam tutarın türetilmesi

Şekil 10'da USD cinsinden iki satış TRY'ye çevrilmiştir: 150 USD → 4.875 TL ve 780 USD → 25.350 TL. `birim_fiyat_try` ve `toplam_tutar_try` gibi türetilmiş sütunlar sayesinde tüm satışlar tek bir para birimi üzerinden karşılaştırılabilir hâle gelmiştir.

2.7. Reddedilen Kayıtlar

Bazı kayıtlar temizleme kurallarıyla kurtarılamayacak kadar bozuktur. Bu kayıtlar atılmaz; sebebiyle birlikte ayrı bir `clean.rejected` tablosuna alınarak izlenebilir kılınır.

```
o -c "SELECT ad, urun, fiyat, adet, sebep FROM clean.rejected ORDER BY
sebep;"
```

ad	urun	fiyat	adet	sebep
Elif Yıldız	Laptop	25000	N/A	Gecersiz adet (sayı değil)
Selin Aydın	Klavye	850	1	Gelecek tarih
Okan Arslan	Mouse	-350	2	Negatif fiyat
Pınar Ak	USB Bellek	180	0	Sıfır adet

(4 rows)

Şekil 11: Kurtarılamayan kayıtların reddedilme sebepleriyle birlikte ayrılması

Şekil 11'de 4 kayıt reddedilmiştir: geçersiz adet ("N/A", sayı değil), gelecek tarih, negatif fiyat ve sıfır adet. Reddedilen kayıtların sebepleriyle saklanması, hem veri kalitesinin izlenmesini hem de gerekirse kaynaktaki sorunların geri bildirilmesini sağlar.

2.8. Önce / Sonra Kalite Karşılaştırması

Temizlemenin etkisini ölçmek için her metrik, temizleme öncesi ve sonrası değerleriyle karşılaştırılır.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d etl_dem
o -c "SELECT o.metrik, o.deger AS once, s.deger AS sonra, GREATEST(o.de
ger-s.deger,0) AS duzeltilen FROM target.quality_profile o JOIN target.
quality_profile s ON o.metrik=s.metrik AND o.asama='ONCE' AND s.asama='
SONRA' ORDER BY o.metrik;"
```

metrik	once	sonra	duzeltilen
eksik_eposta	3	3	0
eksik_sehir	1	1	0
gecersiz_adet	1	0	1
gecersiz_eposta_format	1	0	1
negatif_fiyat	1	0	1
sifir_adet	1	0	1
toplam_ham_kayit	22	22	0

(7 rows)

Şekil 12: Veri kalitesi raporu: her hata metriği için önce / sonra / düzeltilen değerleri

Şekil 12'de görüldüğü gibi geçersiz adet (1→0), geçersiz e-posta formatı (1→0), negatif fiyat (1→0) ve sıfır adet (1→0) sorunlarının tamamı giderilmiştir. Eksik e-posta ve eksik şehir gibi kaynakta bulunmayan veriler ise temizlemeyle üretilmeyeceğinden değişmemiştir. Bu rapor, sürecin somut katkısını sayısal olarak ortaya koyar.

2.9. ETL Log ve Başarı Oranı

Sürecin her adımı, giren ve çıkan satır sayılarıyla birlikte bir log tablosuna kaydedilir. Bu, verinin katmanlar arasında nasıl aktığını izlemeyi sağlar.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d etl_demo -c "SELECT etl_step AS adım, layer AS katman, rows_in AS giris, rows_out AS cikis, rows_rejected AS red FROM target.etl_log ORDER BY id;"
```

adım	katman	giris	cikis	red
EXTRACT	staging	22	22	0
PROFILING	staging	22	0	0
CLEANING	clean	22	16	4
TRANSFORM	clean	16	16	0
LOAD	target	16	16	0

(5 rows)

Şekil 13: ETL süreç akışı: katman katman giren/çıkan/reddedilen satır sayıları

Şekil 13'teki log, akışı şöyle özetler: EXTRACT 22 kayıt almış, PROFILING ile profillemiş, CLEANING aşamasında 22 kayıttan 4'ü reddedilerek 16 kayıt kalmış, TRANSFORM ve LOAD adımları bu 16 kaydı korumuştur. Buna göre başarıyla yüklenen kayıt oranı $16/22 \approx \%72,7$ olarak hesaplanır; geri kalan kısım bilinçli olarak reddedilen bozuk kayıtlardır.

2.10. Idempotent Yükleme (UPSERT)

ETL süreçleri tekrar tekrar çalıştırılabilir. Aynı veri ikinci kez yüklendiğinde mükerrer kayıt oluşmaması için ON CONFLICT ile idempotent (yinelemeye dayanıklı) yükleme kullanılır:

```
-- Aynı id varsa güncelle, yoksa ekle (idempotent yükleme)
INSERT INTO target.sales (id, ad, eposta, toplam_tutar_try)
VALUES (:id, :ad, :eposta, :tutar)
ON CONFLICT (id) DO UPDATE
SET toplam_tutar_try = EXCLUDED.toplam_tutar_try;
```

2.11. Türkçe Karakter Sorunu ve Çözümü

PostgreSQL'in standart initcap fonksiyonu Türkçe "i/I" dönüşümünü hatalı yapar (örneğin "istanbul" → "İstanbul"). Bu sorun, Türkçe kurallarına uyan özel bir tr_initcap fonksiyonu ile çözülmüştür:

```
-- Türkçe karaktere duyarlı baş harf büyütme
CREATE FUNCTION tr_initcap(t text) RETURNS text AS $$
SELECT initcap(replace(replace(t, 'i', 'İ'), 'ı', 'I'));
$$ LANGUAGE sql IMMUTABLE; -- 'istanbul' -> 'İstanbul'
```

2.12. Sonuç ve Değerlendirme

Bu projede iki kaynaktan gelen kirli veri, katmanlı bir ETL mimarisiyle (staging → clean → target) temizlenmiş, dönüştürülmüş ve hedef ambara yüklenmiştir. Veri profillemeye ile sorunlar nicel olarak ölçülmüş, temizleme ile geçersiz adet, negatif fiyat, sıfır adet ve format hataları tamamen giderilmiş, kurtarılamayan 4 kayıt sebebiyle birlikte ayrılmıştır. Önce/sonra kalite raporu ve %72,7'lik yükleme başarısı, sürecin etkisini somut biçimde göstermektedir.

3. Veritabanı Yedekleme ve Otomasyon Çalışması

3.1. Giriş ve Proje Amacı

Bu projenin amacı, yedekleme işlemlerini otomatikleştirmek, yedeklerin düzenli alındığını denetlemek (SLA) ve bir başarısızlık durumunda otomatik uyarı üretmektir. Proje, birinci projedeki backup_demo veritabanını ve backup_history tablosunu kullanır; ayrıca SQL Server Agent'in PostgreSQL'deki karşılığı kurgulanmıştır.

3.2. Tanımlı Yedekleme İşleri

SQL Server Agent Job Nedir?

SQL Server Agent Job, belirli bir zamanlamaya göre otomatik çalışan bir bakım/yedekleme görevidir. PostgreSQL'de yerleşik bir Agent bulunmadığından, bu yapı bir job tablosu (zamanlama ve SLA bilgisini tutar) ile çalıştırma prosedürlerinin birleşimi olarak modellenmiştir.

Projede üç yedekleme işi tanımlanmış, her birine cron zamanlaması ve bir SLA süresi atanmıştır:

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT job_id, job_name, backup_type, schedule_cron, sla_hours FROM backup_job ORDER BY job_id;"
```

job_id	job_name	backup_type	schedule_cron	sla_hours
1	Gunluk Tam Yedek	full	0 2 * * *	24
2	6 Saatlik Fark	diff	0 */6 * * *	6
3	Saatlik Log Yedek	log	0 * * * *	1

(3 rows)

Şekil 14: Tanımlı yedekleme işleri: tür, cron zamanlaması ve SLA süreleri

Şekil 14'teki üç iş şunlardır: Günlük Tam Yedek (cron 0 2 * * *, SLA 24 saat), 6 Saatlik Fark (0 */6 * * *, SLA 6 saat) ve Saatlik Log Yedek (0 * * * *, SLA 1 saat). Her işin kendi SLA süresi, daha sonra denetim adımında ihlal tespiti için kullanılacaktır.

3.3. Job Çalıştırma Prosedürü ve Hata Yakalama

Her iş, çalıştığı anda sonucunu (başarılı/başarısız) backup_history tablosuna yazan bir prosedürle yürütülür. Hatalar EXCEPTION bloğuyla yakalanır ve hata mesajıyla birlikte kaydedilir.

```
-- Job çalıştırma prosedürü (özet)
BEGIN
    -- ... pg_dump çağrısı ...
    UPDATE backup_history SET status='SUCCESS' WHERE id=h_id;
EXCEPTION WHEN OTHERS THEN
    UPDATE backup_history SET status='FAILED', error_msg=SQLERRM WHERE id=h_id;
END;
```



```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT id, job_id, backup_type AS tip, status AS durum, COALESCE(error_msg, '-') AS hata FROM backup_history WHERE job_id IS NOT NULL ORDER BY id DESC LIMIT 5;"
```

id	job_id	tip	durum	hata
36	1	full	SUCCESS	-
35	1	full	FAILED	Disk alanı yetersiz - yedek alınamadı
34	3	log	SUCCESS	-
33	2	diff	SUCCESS	-
32	1	full	SUCCESS	-

(5 rows)

Şekil 15: Job çalışmaları: başarılı (SUCCESS) ve başarısız (FAILED) kayıtlar ve hata mesajı

Şekil 15'te hem SUCCESS hem de FAILED kayıtları görülmektedir. Başarısız çalışmada hata sütununda "Disk alanı yetersiz – yedek alınamadı" mesajı yer alır. Bu, hata yakalama mekanizmasının çalıştığının ve sorunların sessizce kaybolmayıp kayıt altına alındığının kanıtıdır.

3.4. Günlük Yedekleme Raporu

Yedekleme durumunu özetlemek için son 7 günün günlük başarı/başarısızlık sayıları ve başarı yüzdesi raporlanır.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT started_at::DATE AS tarih, COUNT(*) AS toplam, COUNT(*) FILTER (WHERE status='SUCCESS') AS basarili, COUNT(*) FILTER (WHERE status='FAILED') AS basarisiz, ROUND(COUNT(*) FILTER (WHERE status='SUCCESS')::NUMERIC/NULLIF(COUNT(*),0)*100,1) AS basari_pct FROM backup_history WHERE started_at >= CURRENT_DATE-7 GROUP BY started_at::DATE ORDER BY tarih DESC LIMIT 6;"
```

tarih	toplam	basarili	basarisiz	basari_pct
2026-06-04	22	18	4	81.8
2026-06-03	2	2	0	100.0
2026-06-02	2	2	0	100.0
2026-06-01	2	2	0	100.0
2026-05-31	2	2	0	100.0
2026-05-30	2	2	0	100.0

(6 rows)

Şekil 16: Günlük yedekleme raporu: son 7 gün için toplam / başarılı / başarısız sayıları ve başarı yüzdesi

Şekil 16'daki rapora göre çoğu gün %100 başarı sağlanmışken, 04.06.2026 tarihinde 22 yedekten 4'ü başarısız olmuş ve başarı oranı %81,8'e düşmüştür. Bu tür bir durum panosu, yöneticinin bir bakışta sorunlu günü görmesini sağlar.

3.5. Otomatik Zamanlama Üretimi

İşlerin cron zamanlamaları elle yazılmak yerine job tablosundan otomatik üretilir. Böylece bir işin zamanlaması değiştiğinde, pg_cron komutu da kendiliğinden güncellenir.

```
CT format('SELECT cron.schedule(%L, %L, %L);', 'backup_job_'||job_id, schedule_cron,
format('CALL sp_run_backup_job(%s);', job_id)) AS pg_cron_komutu FROM backup_job WH
ERE is_enabled ORDER BY job_id;"

pg_cron_komutu
-----
SELECT cron.schedule('backup_job_1', '0 2 * * *', 'CALL sp_run_backup_job(1);');
SELECT cron.schedule('backup_job_2', '0 */6 * * *', 'CALL sp_run_backup_job(2);');
SELECT cron.schedule('backup_job_3', '0 * * * *', 'CALL sp_run_backup_job(3);');
(3 rows)
```

Şekil 17: Her iş için job tablosundan otomatik üretilen pg_cron zamanlama komutları

Şekil 17'de görüldüğü gibi her iş için cron.schedule('backup_job_N', ...) komutları otomatik oluşturulmuştur. Bu yaklaşım, zamanlama ile iş tanımı arasındaki tutarlılığı garanti eder ve elle yapılan hataları önler.

3.6. SLA Denetimi

SLA Nedir?

SLA (Service Level Agreement / Hizmet Düzeyi Anlaşması), bir hizmetin karşılaması gereken ölçülebilir taahhüttür. Burada her yedekleme işinin "en geç ne kadar sürede bir alınmış olması gerektiğini" belirtir.

Bir iş, SLA süresinden daha uzun süredir alınmamışsa SLA ihlali oluşmuştur ve bu durum otomatik olarak tespit edilmelidir.

```
s -d backup_demo -c "SELECT job_name AS is, sla_hours AS sla_saat, gecen_saat, durum FROM v_sla_audit ORDER BY job_id;
"

is | sla_saat | gecen_saat | durum
-----+-----+-----+-----
Gunluk Tam Yedek | 24 | 0.0 | UYUMLU
6 Saatlik Fark | 6 | 0.1 | UYUMLU
Saatlik Log Yedek | 1 | 5.1 | SLA İHLALI
(3 rows)
```

Şekil 18: SLA denetimi: son yedekten bu yana geçen süre ile SLA süresinin karşılaştırılması

Şekil 18'de Günlük Tam Yedek ve 6 Saatlik Fark işleri SLA'larına uygun (UYUMLU) görünürken, Saatlik Log Yedek işi son 5,1 saattir alınmadığından (SLA 1 saat) "SLA İHLALI" olarak işaretlenmiştir. Bu denetim, yedeklerin yalnızca başarılı olmasını değil, **zamanında** alınmasını da güvence altına alır.

3.7. Otomatik Uyarılar (Trigger)

Bir yedek başarısız olduğunda veya SLA ihlal edildiğinde, ilgili yöneticilere otomatik bildirim gönderilmelidir. Bu, backup_history üzerine kurulan bir trigger ile backup_alert tablosuna uyarı yazılarak sağlanır.

```
-- Başarısızlıkta otomatik uyarı üreten trigger (özet)
IF NEW.status = 'FAILED' THEN
  INSERT INTO backup_alert(alert_type, severity, notified_to, message)
  VALUES ('BACKUP_FAILED', 'CRITICAL',
    'dba@sirket.com, sysadmin@sirket.com',
    'Yedekleme BASARISIZ oldu (history_id='||NEW.id||')');
END IF;
```

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT a.severity AS oncelik, a.alert_type AS tip, j.job_name
AS is, LEFT(a.message,45) AS mesaj, a.notified_to AS bildirildi FROM backup_alert a LEFT JOIN backup_job j ON a.job_id=j.job_id WHERE NOT a.
is_resolved ORDER BY CASE a.severity WHEN 'CRITICAL' THEN 1 ELSE 2 END;"
```

oncelik	tip	is	mesaj	bildirildi
CRITICAL	BACKUP_FAILED	Gunluk Tam Yedek	Yedekleme BASARISIZ oldu (history_id=35, tip=	dba@sirket.com, sysadmin@sirket.com
WARNING	SLA_BREACH	Saatlik Log Yedek	SLA IHLALI: "Saatlik Log Yedek" için son yede	dba@sirket.com

(2 rows)

Şekil 19: Otomatik uyarılar: önceliğe göre yönetici yönlendirmesi (CRITICAL ve WARNING)

Şekil 19'da iki uyarı görülmektedir: başarısız yedek için **CRITICAL** seviyesinde bildirim hem dba'ya hem de sysadmin'e gönderilirken; SLA ihlali için **WARNING** seviyesinde bildirim yalnızca dba'ya yönlendirilmiştir. Önceliğe göre farklı yöneticilere yönlendirme, SQL Server'daki Operator/Alert yapısının karşılığıdır.

3.8. Genel Denetim Özeti

Tüm denetim sonuçları tek bir fonksiyon çağrısıyla özetlenebilir; böylece sistemin genel sağlığı bir bakışta görülür.

```
furkanyilmaz@MacBook-Pro-2 BLM4522_proje3 % psql -U postgres -d backup_demo -c "SELECT * FROM fn_audit_summary();"

```

denetim_basligi	sonuc
Toplam aktif job	3
SLA ihlali olan	1
Yedegi hic alınmamis	0
Son 24 saatte basarisiz	4
Genel durum	DIKKAT GEREKLI

(5 rows)

Şekil 20: fn_audit_summary() çıktısı: aktif job, SLA ihlali, başarısızlık sayıları ve genel durum

Şekil 20'deki `fn_audit_summary()` fonksiyonu şu özeti üretir: toplam 3 aktif job, 1 SLA ihlali, hiç yedeği alınmamış iş 0, son 24 saatte 4 başarısızlık ve genel durum "DİKKAT GEREKLİ". Bu tek satırlık özet, yöneticinin sistemin durumunu hızlıca değerlendirmesini sağlar.

3.9. Gerçek Job Runner Scripti

Kurgunun gerçekten çalıştığını göstermek için, prosedürleri tetikleyip gerçek bir `pg_dump` yedeği alan çalıştırılabilir bir kabuk scripti (`backup_runner.sh`) yazılmıştır. Bu script, PowerShell tabanlı SQL Server otomasyonunun `bash` karşılığıdır.

```
# backup_runner.sh – gerçek job çalıştırıcı (özet)
psql -c "CALL sp_run_backup_job($JOB_ID);"      # history kaydını aç + çalıştır
pg_dump -Fc backup_demo -f backups/full_job${JOB_ID}_${date +%Y%m%d}.dump
echo "BASARILI: yedek alındı ($(du -h $DUMP))"   # sonucu bildir
```

```
1/scripts/backup_runner.sh 1

=====
Job #1 calistiriliyor: Thu  4 Jun 2026 20:22:35 +03
=====
NOTICE:  Job 1 basladi (history_id=37)
History kaydi acildi: 37 (tip=full)
NOTICE:  Job tamamlandi (history_id=37): SUCCESS
sp_finish_backup
-----

(1 row)

BASARILI: /Users/furkanyilmaz/BLM4522_proje3/backup_sql/backups/full_job1_20260604_202235.dump (1 MB)
Job #1 tamamlandi.
furkanyilmaz@MacBook-Pro-2: ~ %
```

Şekil 21: `backup_runner.sh` çalışması: gerçek bir `pg_dump` yedeğinin alınması ve `history`'e işlenmesi

Şekil 21, scriptin Job #1'i çalıştırdığını, `backup_history` tablosunda 37 numaralı kaydı açtığını, gerçek bir `full_job1_...` dump dosyası (1 MB) ürettiğini ve sonucu SUCCESS olarak işlediğini göstermektedir. Böylece tanım, zamanlama, çalıştırma, denetim ve uyarı adımları uçtan uca bağlanmış olur.

3.10. SQL Server Agent – PostgreSQL Karşılıkları

Otomasyon projesinde kullanılan kavramların SQL Server ve PostgreSQL karşılıkları:

SQL Server Kavramı	PostgreSQL Karşılığı
Agent Job	backup_job tablosu + sp_run_backup_job prosedürü
Job Schedule	schedule_cron + pg_cron
Job History	backup_history tablosu
Operator / Alert	backup_alert + trigger (önceliğe göre yönlendirme)
Maintenance Plan Report	fn_audit_summary() / v_sla_audit görünümü
PowerShell otomasyon	backup_runner.sh (bash)

3.11. Sonuç ve Değerlendirme

Bu projede yedekleme işleri otomatikleştirilmiş, çalışma geçmişi kaydedilmiş, SLA denetimiyle gecikmeler tespit edilmiş ve başarısızlık/ihlal durumlarında önceliğe göre otomatik uyarılar üretilmiştir. Gerçek bir kabuk scriptiyle alınan canlı yedek, kurgunun teoride kalmadığını gösterir. Sonuç olarak yedekleme süreci, izlenebilir ve kendini denetleyen bir sisteme dönüştürülmüştür.

Genel Sonuç ve Değerlendirme

Bu raporda sunulan üç proje, bir veritabanı yönetim yaşam döngüsünün birbirini tamamlayan üç temel halkasını oluşturur. İlk proje verinin **güvenliğini** ele alır: doğru yedekleme stratejisi ve felaketten kurtarma planı sayesinde, kazara silinen veri eksiksiz geri getirilebilmiştir. İkinci proje verinin **kalitesini** ele alır: ETL süreçleriyle kirli ve tutarsız veri, analize hazır güvenilir bir hâle dönüştürülmüştür. Üçüncü proje ise bu işlemlerin **sürdürülebilirliğini** sağlar: otomasyon, SLA denetimi ve uyarı mekanizmalarıyla yedekleme süreci insan müdahalesine bağımlı olmaktan çıkıp kendini izleyen bir sisteme dönüşmüştür.

Bu üç halka birlikte değerlendirildiğinde ortaya bütüncül bir resim çıkar: kaliteli veri (ETL) düzenli olarak yedeklenir (yedekleme/kurtarma) ve bu yedeklemenin zamanında ve başarılı yapıldığı sürekli denetlenir (otomasyon). Bir halkanın eksikliği diğerlerini de zayıflatır; örneğin yedek alınsa bile denetlenmiyorsa, bir başarısızlık fark edilmeden veri kaybına yol açabilir. Projeler ayrıca SQL Server kavramlarının PostgreSQL'deki karşılıklarını da göstererek, farklı veritabanı sistemleri arasındaki ilkesel sürekliliği ortaya koymuştur.

Tüm çalışmalar PostgreSQL 14 üzerinde, Türkçe açıklamalı ve numaralandırılmış SQL betikleriyle gerçekleştirilmiş; proje kaynak kodları github.com/furkanyilmaz/BLM4522 adresinde paylaşılmıştır.